Distributed Job Scheduling Platform

A production-inspired platform for reliable asynchronous background jobs: REST API, horizontally-scalable worker fleet, retries with configurable backoff, dead-letter queue, cron schedules, workflow DAGs, rate limiting, queue sharding, event-driven execution, WebSocket live updates and AI failure diagnoses.

Anoop Mahesh · maheshanoop05@gmail.com

Source code: https://github.com/kraken222/distributed-job-scheduler

Stack: Node 22 · TypeScript · Express · SQLite (WAL) · React · Vite · Vitest

July 2026

Deliverables in this document

1 Source code & setup instructions

The complete source code, documentation and test suite live at https://github.com/kraken222/distributed-job-scheduler (public, branch `main`). The repository is self-contained: no Docker, no database server — storage is embedded SQLite in WAL mode, chosen so a reviewer can run the full distributed system with nothing but Node.js ≥ 20 (the schema and claim queries are written portably, with the Postgres migration path documented).

Repository layout

```
job-scheduler/
+-- server/           # API + worker service (TypeScript, Node 22)
|   +-- src/api/     # Express app: routes, auth, validation, WebSocket hub
|   +-- src/core/    # domain logic: claims, retry, scheduler, reaper, events, summaries
|   +-- src/db/      # SQLite connection, versioned migrations, seed
|   +-- src/worker/  # worker runtime + job handlers
|   +-- tests/       # Vitest suite (82 tests, 14 files)
+-- web/             # React dashboard (Vite)
+-- docs/            # architecture, ER diagram, API reference, design decisions
```

Quick start

```
# 1. API + coordinators (cron scheduler, reaper, failure summarizer, WebSocket hub)
cd server && npm install
npm run seed    # demo org, queues, workflow DAG, triggers (demo@example.com / demo1234)
npm run api     # → http://localhost:4000

# 2. Workers — start as many as you like, each in its own terminal
npm run worker
WORKER_NAME=worker-2 npm run worker         # a second one
WORKER_SHARDS=0,2 npm run worker            # pinned to shards 0 and 2

# 3. Dashboard
cd web && npm install && npm run dev    # → http://localhost:5173

# 4. Tests + type checks
cd server && npm test && npm run typecheck
```

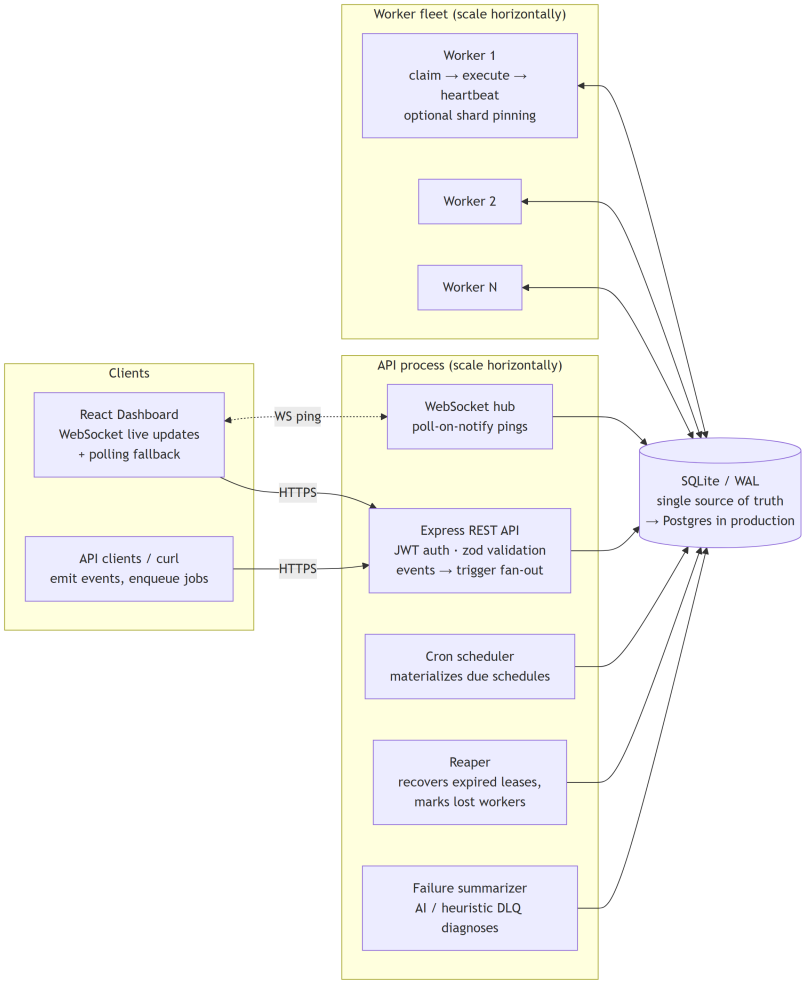
The seed demonstrates every feature out of the box: an ETL **workflow DAG**, a **rate-limited** queue (5 starts / 10 s), a **sharded** per-tenant queue, a **user.signup event trigger**, and a job that dead-letters and receives an automatic **failure diagnosis**.

Configuration (environment variables)

Variable	Default	Purpose
PORT	4000	API port
DATABASE_FILE	<repo>/data/scheduler.db	SQLite file shared by API & workers
JWT_SECRET	dev value	set in production
WORKER_NAME / WORKER_CONCURRENCY	worker-<pid> / 5	worker identity and parallelism
WORKER_SHARDS	all shards	comma-separated shard pinning, e.g. 0,2
LEASE_MS / WORKER_STALE_MS	30000 / 15000	claim lease and lost-worker threshold
ANTHROPIC_API_KEY	unset	enables AI failure summaries (deterministic heuristic otherwise)

2 System architecture

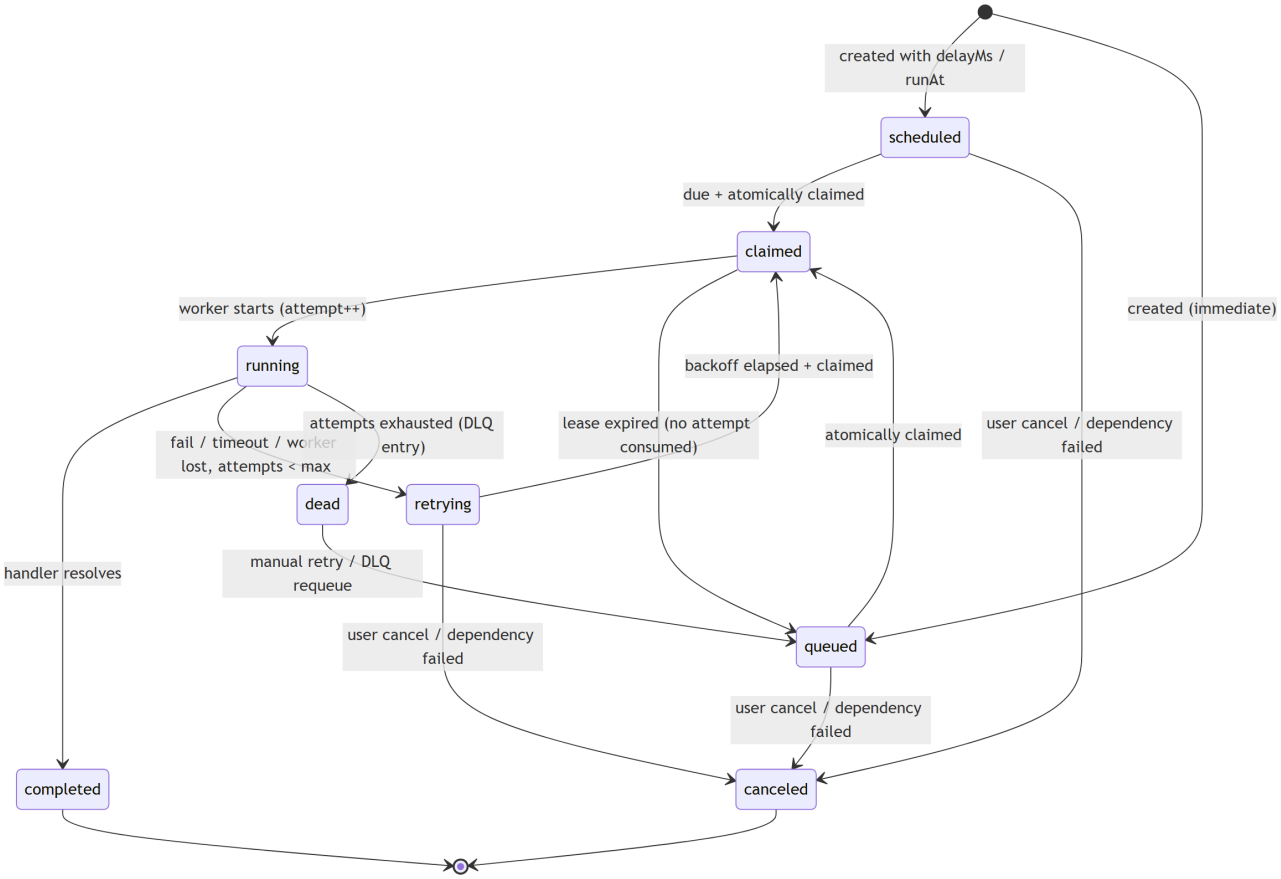
Three deployable units share one database. There is deliberately **no message broker**: the database is the queue. Workers pull work with atomic claims; job state and business data commit together, which removes the classic dual-write failure mode (job row committed but message lost). This is the same pattern used in production by Postgres-backed queues (Oban, Solid Queue, pg-boss).



Architecture: clients, horizontally-scalable API process and worker fleet around one transactional store.

Component	Responsibility
Express REST API	JWT auth, zod validation, pagination/filtering, structured errors; event → trigger fan-out commits atomically with the event record
WebSocket hub	authenticated /api/ws; pushes debounced “changed” pings (poll-on-notify) — clients refetch via REST, so tenancy rules are never duplicated
Cron scheduler	materializes due schedules into job rows; compare-and-set on next_run_at + idempotency key ⇒ N API instances never double-fire
Reaper	marks stale workers lost; expired leases: claimed → back to queue (no attempt burned), running → normal retry/DLQ path
Failure summarizer	attaches AI/heuristic diagnoses to new DLQ entries; idempotent, multi-instance safe
Worker fleet	registers, heartbeats (renewing leases), claims atomically up to its concurrency, executes with per-job timeouts (AbortSignal), drains gracefully on SIGTERM

Job lifecycle



Full lifecycle with retry/backoff, dead-letter and cancellation branches.

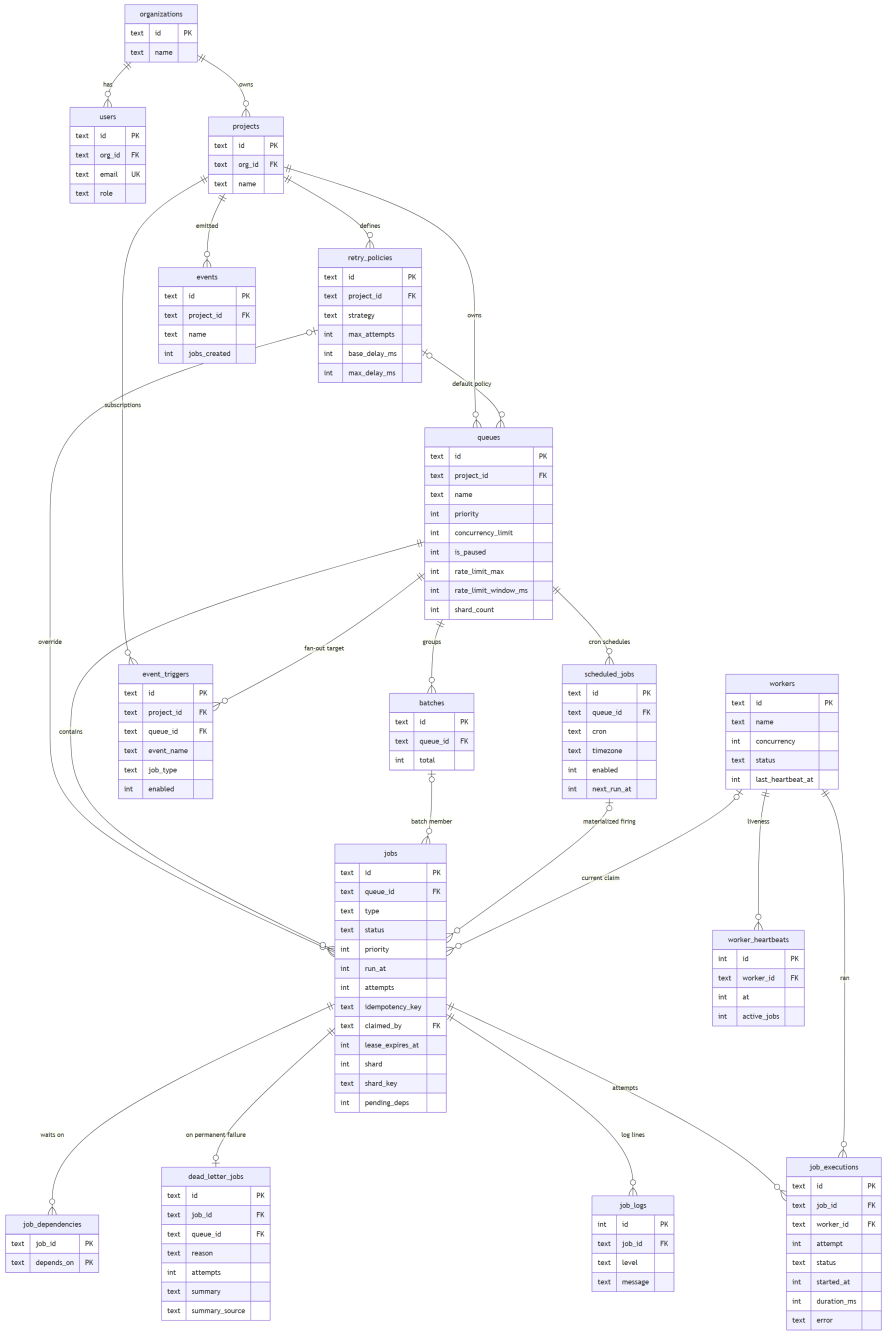
Workflow dependencies overlay this machine: a job with incomplete parents stays queued but is invisible to claims until its pending-dependency counter reaches zero (each completing parent decrements it). A parent that dies or is canceled recursively cancels its descendants — a DAG never half-runs on missing inputs.

Atomic claiming (no duplicate execution)

Claiming is one `BEGIN IMMEDIATE` transaction: a CTE computes per-queue free slots as **MIN(concurrency headroom, rate-limit tokens)**, candidates are filtered to runnable jobs only (due, dependencies complete, on the worker's shards) and ranked by queue priority → job priority → FIFO with a window function so a claim never exceeds any queue's capacity; the UPDATE re-checks status, making the transition compare-and-set. SQLite serializes writers across processes — the same guarantee `SELECT … FOR UPDATE SKIP LOCKED` provides on Postgres, to which the query maps 1:1. Because limits, rate tokens and the dependency gate are evaluated under the same write lock that performs the claim, they are exact across the entire fleet.

3 Database design (ER diagram)

Schema ships as versioned, forward-only migrations (PRAGMA user_version) applied idempotently by every process. All timestamps are epoch milliseconds. Primary keys are prefixed random IDs (job_…, que_…) — self-describing in logs, non-enumerable across tenants; high-volume append-only tables (logs, heartbeats) use rowid keys instead.



ER diagram — 14 tables; key columns shown (full schema in docs/er-diagram.md and src/db/schema.ts).

Keys, cascades and normalization

Relation	On delete	Rationale
org → users/projects → queues → jobs → executions/logs/dependencies	CASCADE	tenant data is a strict ownership tree; no orphans
queue/job → retry_policy	SET NULL	deleting a policy must not delete jobs; they fall back to the system default at resolution time
job → batch / scheduled_job	SET NULL	history metadata, not ownership
jobs.claimed_by → workers	SET NULL	a deregistered worker must never take jobs down with it

The schema is 3NF where it matters (retry policies are referenced entities; queue config lives only on queues; outcomes live only on job_executions) with **measured denormalizations**: jobs.status/attempts/last_error keep the hottest queries off aggregates; **jobs.pending_deps** turns the dependency gate into an indexed integer comparison instead of a DAG walk on every worker poll — maintained inside the same transactions that transition parent state, so it cannot drift; DLQ rows snapshot reason/attempts so triage needs no joins.

Indexes (each exists for a named query)

Index	Serves
idx_jobs_claim (queue_id, status, run_at, priority)	the worker claim scan — hottest query in the system
idx_jobs_status (status, run_at)	reaper scan and global status counts
UNIQUE (queue_id, idempotency_key) WHERE NOT NULL	O(log n) enqueue dedupe; partial ⇒ keyless jobs cost nothing
idx_deps_parent (depends_on)	completion fan-out and failure cascade
idx_executions_started (started_at)	sliding-window rate-limit count inside the claim transaction
idx_triggers_lookup (project_id, event_name, enabled)	emit-time trigger matching
idx_schedules_due (enabled, next_run_at)	scheduler tick is a range scan, not a table scan
idx_executions_finished / idx_dlq_queue / idx_jobs_list	throughput buckets, DLQ listing, job explorer pagination

WAL mode lets dashboard reads proceed while workers write; busy_timeout queues writers instead of erroring; worker heartbeat history is bounded (last 100/worker); unique constraints double as business rules (project and queue names unique per scope, one email per user).

4 API documentation

Base URL `http://localhost:4000/api`. Everything except /auth/* and /health requires `Authorization: Bearer <jwt>`. Errors are always structured (`{ error: { code, message, details } }`); another organization's resources return **404, not 403** (no existence leaks). Pagination: `?page&limit` → `{ data, pagination }`. RBAC: destructive operations are admin-only. Full reference with request/response shapes: `docs/api.md`.

Endpoint	Purpose
POST /auth/register · /auth/login · GET /auth/me	org + admin creation, login (uniform 401), current identity
CRUD /projects · GET :id/overview · :id/throughput	projects, live health totals, densified throughput buckets
CRUD /projects/:id/queues · /queues/:id	priority, concurrencyLimit, retryPolicyId, rateLimitMax + rateLimitWindowMs (set together), shardCount
POST /queues/:id/pause · /resume · GET :id/stats	pause/resume claiming; depth, running, per-hour stats, oldest wait
POST /queues/:id/jobs	immediate / delayed (delayMs) / scheduled (runAt); priority, timeout, retry override, idempotencyKey, dependsOn[], shardKey
POST /queues/:id/batches · /workflows	batch (≤1000 jobs); workflow = atomic DAG with sibling keys, cycle detection → 400
GET /queues/:id/jobs?status&type&search&page	job explorer: filterable, searchable, paginated
GET /jobs/:id · /executions · /logs · /dependencies	full job, per-attempt history, log lines, parents/children with statuses
POST /jobs/:id/cancel · /retry	cancel (cascades to dependents) · retry dead/canceled/completed or fast-forward backoff
CRUD /queues/:id/schedules · /schedules/:id	cron (validated, timezone-aware); cadence edits recompute next firing
POST /projects/:id/events · GET events · CRUD triggers	emit event → atomic fan-out via enabled triggers; audit list with jobs_created
GET /workers · /workers/:id/heartbeats	fleet observability: status, slots, lifetime counters, liveness history
POST /dlq/:id/requeue	re-queue buried job (attempts reset); 409 if already requeued; entries carry summary + summary_source
GET /api/ws?token=<jwt>	WebSocket: hello on connect, debounced { type: “changed” } pings; 401 before upgrade on bad token

5 Design decisions & major trade-offs

Guiding principle: **correctness under concurrency first, feature count second**. Each decision below names the trade-off taken; the full document (17 sections) is `docs/design-decisions.md`.

Decision	Why / trade-off
Database-as-queue, no broker	one transactional source of truth kills the dual-write problem; cost: polling latency (≤500 ms) and DB write load — acceptable until ~10^4 jobs/min, then shard/archive
SQLite now, Postgres path documented	reviewer runs everything with zero infrastructure; writers serialize across processes so the no-duplicate-claim guarantee is identical to FOR UPDATE SKIP LOCKED; cost: single-writer ceiling
Lease-based liveness	“stopped renewing its lease” is observable, “process died” is not; stale writers are rejected by guarded transitions; cost: up to lease + tick (~35 s) recovery delay
At-least-once + idempotency, not exactly-once	exactly-once side effects are impossible in a distributed system; the platform provides the achievable halves: exactly-once enqueue (unique key) and exactly-once cron firing (CAS)
Attempts count at running, not claim	a claim that never started did nothing — a flapping worker must not dead-letter jobs that never ran; a lost running job must consume budget (side effects may have happened)
Retry policies as referenced entities	operator-tunable config, not values frozen onto job rows; resolution job → queue → system default; fixed/linear/exponential with ±10% jitter against retry herds
Fleet-wide limits inside the claim transaction	per-worker self-limiting cannot express global caps; same write lock ⇒ no TOCTOU, limits are exact
Rate limit counts execution starts	downstream APIs care how often they are hit; claimed-but-unstarted jobs also consume tokens, closing the burst-claim gap; retries consume tokens deliberately
Dependency gate as a counter	claim-time NOT EXISTS subquery re-walks edges on every poll of every worker; the counter moves cost to completion time and cannot drift (same transactions)
Sharding = partitioning + voluntary pinning	same key ⇒ same shard enables dedicated workers per tenant; unpinned workers see all shards, so a dead pinned worker degrades latency, never strands a shard
Events: atomic fan-out, exact-name matching	event + jobs commit together (never half-processed); wildcard routing skipped deliberately — it is the first version of every rules engine that ends up needing its own debugger
WebSockets push pings, not data	data-push would duplicate tenancy/pagination in a second protocol; ping + REST refetch keeps authorization in one place and degrades to polling for free
AI summaries async with deterministic floor	LLM calls have no place in a failure transaction; heuristic fallback works offline and on any AI error; row records which engine wrote it
Cooperative cancellation only	Node cannot safely preempt async code; pretending to kill a running job while side effects continue would report a lie — leases guarantee system-level recovery instead
Tenancy as SQL in one module	every lookup joins to the caller's org; scattered if-checks are where multi-tenant systems leak; 404 (not 403) avoids confirming existence

6 Reliability & concurrency — verified behaviour

Claims of correctness are backed by tests and by measurements against the running system:

Property	Evidence
No duplicate execution	two workers claiming from the same 10-job queue receive disjoint sets (test); every transition guarded by status + claimed_by so stale writers no-op
Fleet-wide concurrency limits	queue with limit 3: first worker claims 3, second claims 0 (test)
Exact rate limiting	measured live: a 12-job queue limited to 5 starts/10 s drained in buckets of exactly 5 / 5 / 2 over 20.5 s
Dependency gating	diamond DAG executes level-by-level; child invisible until every parent completed; dead parent cancels the descendant subtree (tests + live run)
Stable shard placement	same shard key ⇒ same shard every time; pinned worker claims only its shards, unpinned worker mops up the rest (tests + live run)
Crash recovery	killed worker: claimed jobs return to queue without burning an attempt; running jobs close as lost and follow the retry policy; execution history records it (tests)
Cron exactness	CAS on next_run_at + idempotency key: N schedulers race, exactly one job per firing (test)
Graceful shutdown	SIGTERM: worker drains in-flight jobs, aborts stragglers at deadline, marks itself offline (integration test)
WebSocket integrity	bad JWT rejected with 401 before upgrade; a 25-notification burst coalesces into ≤3 pings (tests)
Timeout enforcement	handlers race a per-job timer; AbortSignal lets them stop cooperatively; timed-out attempts recorded distinctly (tests)

7 Frontend & UX

Responsive React dashboard (light professional theme — Inter, teal accent, dark slate sidebar) covering: project overview with throughput chart and queue health; queue detail with live stats, full configuration (priority, concurrency, retry policy, rate limit, shards) and an enqueue form (immediate/delayed/scheduled/batch, shard key); a job explorer with search, status filters and pagination; job detail with payload/result, retry history, logs and the **workflow dependency graph**; workers with heartbeat history; DLQ with one-click requeue and **automatic failure diagnoses**; cron schedule management; an **Events page** (emit events, manage triggers, audit fan-out); and retry-policy management with backoff previews. **Live updates ride the WebSocket** (a sidebar indicator shows Live vs Polling) and every view degrades gracefully to visibility-aware polling. The UI holds no privileged backdoors — it can do exactly what the public API allows.

8 Automated tests (82 tests, 14 files)

Run with `cd server && npm test` (Vitest; fresh in-memory SQLite per test — fast, isolated, deterministic). Tests exercise the real state machine end to end, not mocks.

Suite	What it proves
claims.test.ts	atomic claiming: no duplicates across workers, pause handling, priority/FIFO ordering, concurrency caps, lease setting
lifecycle.test.ts	full transitions incl. guarded no-ops for stale workers, cancellation rules, manual retry semantics
retry.test.ts	backoff math for fixed/linear/exponential incl. jitter bounds and caps
reaper.test.ts	lost workers, lease expiry for claimed vs running, attempt accounting, DLQ path
scheduler.test.ts	cron materialization, CAS race (N schedulers ⇒ 1 job), invalid-cron quarantine
dependencies.test.ts	claim gating, completion fan-out, dead/canceled cascade, DAG creation, cycle/self/duplicate-key rejection, atomicity
ratelimit.test.ts	window budget incl. claimed-but-unstarted jobs, sliding expiry, MIN(concurrency, tokens), queue isolation
sharding.test.ts	deterministic FNV-1a placement, keyless spread, pinned vs unpinned claiming, WORKER_SHARDS parsing
events.test.ts	trigger fan-out, payload envelope merging, disabled triggers, cross-project isolation
summary.test.ts	heuristic classifier per failure family; summarizer idempotency on real DLQ entries
ws.test.ts	JWT rejection before upgrade, hello/changed protocol, burst debouncing (real sockets)
api.test.ts	auth flows, validation error shapes, RBAC, tenancy 404s, pagination/filtering
worker.test.ts	integration: real worker executes, retries, respects concurrency, drains gracefully
stats.test.ts	queue stats and throughput bucketing (incl. the REAL-binding CAST regression)

9 Bonus features — all eight implemented

Bonus feature	Implementation
Workflow dependencies	job_dependencies + pending_deps claim gate; POST /workflows creates DAGs atomically with cycle rejection; failed/canceled parents cascade-cancel descendants
Rate limiting	per-queue sliding-window cap on execution starts, enforced exactly inside the claim transaction, fleet-wide
Distributed locking	the IMMEDIATE claim transaction is the lock (equivalent to FOR UPDATE SKIP LOCKED); optimistic CAS locks schedule firing
Queue sharding	shard_count per queue; FNV-1a placement by shard key; WORKER_SHARDS pins workers to shard subsets
Event-driven execution	events + triggers with atomic fan-out and audit trail; managed from the dashboard's Events page
WebSocket live updates	authenticated /api/ws, debounced poll-on-notify pings, data_version pragma for cross-process detection, polling fallback
Role-based access control	admin/member roles in JWT; requireRole middleware gates destructive operations
AI failure summaries	async summarizer: Claude via ANTHROPIC_API_KEY with a deterministic heuristic classifier as offline/error fallback; source recorded per entry

10 Evaluation criteria — where to look

Criterion	Marks	Where it is addressed
System architecture	20	Section 2; docs/architecture.md; three-process design, coordinator loops, poll-on-notify live updates
Database design	20	Section 3; docs/er-diagram.md; src/db/schema.ts — keys, cascades, partial indexes, measured denormalization
Backend engineering	20	server/src — layered core/api/worker/db modules; validation, structured errors, logging, migrations, seed
Reliability & concurrency	15	Section 6; core/claims.ts, reaper.ts — atomic claims, leases, at-least-once + idempotency, exact fleet-wide limits
Frontend & UX	10	Section 7; web/src — live dashboard, job explorer, DLQ triage with diagnoses, Events page, workflow graph
API design	5	Section 4; docs/api.md — consistent REST, pagination, filtering, structured errors, 404-over-403 tenancy
Documentation	5	README + four docs (architecture, ER, API, design decisions) + this submission document
Testing	5	Section 8; 82 automated tests over every critical property, incl. concurrency races and real WebSockets

Repository: https://github.com/kraken222/distributed-job-scheduler · **Demo login:** demo@example.com / demo1234 · **Contact:** maheshanoop05@gmail.com